

OPTION B INSTITUTE FAMILY

M05 - PYTHON FOR DATA ENGINEERING

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

15 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE05-01 - CSV properly: parse, inspect, preserve raw	5
DE05-02 - JSON and JSONL from zero	8
DE05-03 - Paths, folders and deterministic batch files	10
DE05-04 - pandas profiling before transformation	12
DE05-05 - Cleaning with pandas and rejected rows	14
DE05-06 - pandas merge, cardinality and aggregation	16
DE05-07 - Data validation: record, batch and relationship rules	19
DE05-08 - HTTP and REST APIs from zero	22
DE05-09 - Calling APIs safely with requests	25
DE05-10 - Pagination and authentication without leaking secrets	28
DE05-11 - Retries, rate limits and partial failure	31
DE05-12 - Python + PostgreSQL with SQLAlchemy	34
DE05-13 - Configuration, environment variables and secret hygiene	37
DE05-14 - Logging, testing and reusable pipeline code	40
DE05-15 - Integrated Python data-engineering pipeline	43
Module mastery checklist + revision quick reference	46

Prerequisite / tool map

Use Python for files, pandas, APIs, PostgreSQL, validation, logging and reusable pipeline code.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE05-01 - CSV properly: parse, inspect, preserve raw

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Parse CSV deliberately, inspect its contract, and preserve raw input before transformation.
- TEACH IT TO YOURSELF
- A CSV file is text with a tabular convention, not a magically typed table. Preserve the received bytes or immutable source copy so later
- debugging can distinguish source from transformation.
- WORKED DEMONSTRATION
- Use `csv.DictReader` or `pandas read_csv` for inspection. Record filename, row count, columns, encoding assumption and a
- SHA-256/source-copy note before changing values.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Reading and Writing CSV Files in Python Full focused CSV lesson. Watch first; close video before the Teaching Book.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Reading and Writing CSV Files in Python Full focused CSV lesson. Watch first; close video before the Teaching Book. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Inspect `orders_good.csv`. Save raw row count, field names and first/last `order_id`. Copy it to an immutable `raw_received` folder without editing. Inspect `orders_good.csv`. Save raw row count, field names and first/last `order_id`. Copy it to an immutable `raw_received` folder without editing.

8. Expected result / what you should see

- Row count and field list are explicit; raw copy hash equals source hash; no transformation overwrites raw.
- COMMON FAILURE / RECOVER
- Failure to avoid: Opening CSV in Excel and saving it back before preserving the original can silently change formatting/types. Recovery:
 - stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the
 - independent validation. Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED

- Receive data/retest/shipments_retest.csv. Save its SHA-256 and row/field controls before doing any cleanup. Create your own
- byte-identical receipt copy under data/work/retest_received/.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Given orders_new.csv, produce a source receipt note and prove the preserved raw copy is byte-identical to the supplied source.

13. Validate

- Row count and field list are explicit; raw copy hash equals source hash; no transformation overwrites raw.
- COMMON FAILURE / RECOVER
- Failure to avoid: Opening CSV in Excel and saving it back before preserving the original can silently change formatting/types. Recovery:
 - stop publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the
 - independent validation. Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Receive data/retest/shipments_retest.csv. Save its SHA-256 and row/field controls before doing any cleanup. Create your own
- byte-identical receipt copy under data/work/retest_received/.

14. Teach it back

- Without the book/video open, explain csv properly: parse, inspect, preserve raw in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-01 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-02

DE05-02 - JSON and JSONL from zero

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Read and write JSON and JSONL while explaining their different record shapes.
- TEACH IT TO YOURSELF
- JSON can represent one nested value/document. JSONL stores one JSON value per line, useful for append/stream-style records and
 - easy line-level recovery.
- WORKED DEMONSTRATION
- Read `orders_sample.json` with `json.load`; write a few records to `.jsonl` one record per line; read back line-by-line.

3. Prerequisite / starting state

Prerequisite: DE05-01 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Working with JSON Data using the `json` Module Full focused JSON lesson. Teaching Book adds JSONL and line-level recovery.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Working with JSON Data using the `json` Module Full focused JSON lesson. Teaching Book adds JSONL and line-level recovery. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Convert six sample JSON records to JSONL and prove every line parses independently. Convert six sample JSON records to JSONL and prove every line parses independently.

8. Expected result / what you should see

- Every JSONL line parses; IDs/count reconcile; nested data meaning is not flattened accidentally.
- COMMON FAILURE / RECOVER
- Failure to avoid: Treating a JSON array file as if each physical line were a full JSON record. Recovery: stop publishing, restate the
 - input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
 - a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Convert `data/retest/shipments_sample.json` into JSONL one object per line, then reconstruct the list. Prove `shipment_id` count/set is
 - identical.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create a new three-record nested JSON source, convert to JSONL, then reconstruct the list and compare IDs/counts.

13. Validate

- Every JSONL line parses; IDs/count reconcile; nested data meaning is not flattened accidentally.
- COMMON FAILURE / RECOVER
- Failure to avoid: Treating a JSON array file as if each physical line were a full JSON record. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Convert data/retest/shipments_sample.json into JSONL one object per line, then reconstruct the list. Prove shipment_id count/set is
- identical.

14. Teach it back

- Without the book/video open, explain json and jsonl from zero in 60-90 seconds: what one input row/response represents, the critical
- failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-02 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-03

DE05-03 - Paths, folders and deterministic batch files

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Use pathlib and deterministic folder/file naming so batch runs are reproducible.
- TEACH IT TO YOURSELF
- Paths are part of pipeline behavior. Derive paths from project root/config and name run outputs from explicit business/run keys, not
- random Downloads locations.
- WORKED DEMONSTRATION
- Use Path(__file__).resolve() and project-relative directories; create received/accepted/rejected/output/state folders; construct a
- deterministic batch filename.

3. Prerequisite / starting state

Prerequisite: DE05-02 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_REFERENCE: Python 3.14 pathlib documentation Basic use, Path construction/resolution, directory creation and file I/O. No weak tutorial video invented.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_REFERENCE: Python 3.14 pathlib documentation Basic use, Path construction/resolution, directory creation and file I/O. No weak tutorial video invented. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Build batch_path(run_date, source_name) and show the same inputs produce the same path. Build batch_path(run_date, source_name) and show the same inputs produce the same path.

8. Expected result / what you should see

- Same logical run keys produce same intended paths; no personal absolute path is required.
- COMMON FAILURE / RECOVER
- Failure to avoid: Hard-coding C:/Users/.../Downloads makes the pipeline non-portable. Recovery: stop publishing, restate the input/output
- contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by
- dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- For source=shipments and run_date=2026-08-15, derive landing/accepted/rejected paths twice from project root/config. Prove identical

- inputs produce identical paths and no C:/Users/... dependency appears.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

For a new source and run date, create the expected landing/output paths from config and prove rerunning path construction makes no duplicate/random folder.

13. Validate

- Same logical run keys produce same intended paths; no personal absolute path is required.
- COMMON FAILURE / RECOVER
- Failure to avoid: Hard-coding C:/Users/.../Downloads makes the pipeline non-portable. Recovery: stop publishing, restate the input/output
- contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by
- dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- For source=shipments and run_date=2026-08-15, derive landing/accepted/rejected paths twice from project root/config. Prove identical
- inputs produce identical paths and no C:/Users/... dependency appears.

14. Teach it back

- Without the book/video open, explain paths, folders and deterministic batch files in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-03 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-04

DE05-04 - pandas profiling before transformation

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Profile a pandas batch before transforming it.
- TEACH IT TO YOURSELF
- Profiling answers what arrived: rows, columns, types, missingness, candidate-key uniqueness and simple ranges. Transformation should
- start only after these controls are recorded.
- WORKED DEMONSTRATION
- Load orders_good.csv, inspect shape/dtypes/nulls/duplicates and basic numeric/date ranges before adding business logic.

3. Prerequisite / starting state

Prerequisite: DE05-03 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 4:22:41-4:47:03 - pandas introduction and inspection. 1.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 4:22:41-4:47:03 - pandas introduction and inspection. 1. STARTING STATE Prerequisite: DE05-03.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56-5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Create a profiling dictionary and save it as JSON evidence. Create a profiling dictionary and save it as JSON evidence.

8. Expected result / what you should see

- Profile is based on observed source; row/key/null/type controls are saved separately from cleaning.
- COMMON FAILURE / RECOVER
- Failure to avoid: Cleaning while profiling destroys evidence of what actually arrived. Recovery: stop publishing, restate the input/output
- contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by
- dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Profile shipments_retest.csv before cleaning. Save rows, columns, dtypes, missing counts, duplicate shipment_id count and suspicious
- date/value/quantity observations.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Profile orders_profile_variant.csv and flag at least three suspicious properties without fixing them yet.

13. Validate

- Profile is based on observed source; row/key/null/type controls are saved separately from cleaning.
- COMMON FAILURE / RECOVER
- Failure to avoid: Cleaning while profiling destroys evidence of what actually arrived. Recovery: stop publishing, restate the input/output
- contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide a defect by
- dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Profile shipments_retest.csv before cleaning. Save rows, columns, dtypes, missing counts, duplicate shipment_id count and suspicious
- date/value/quantity observations.

14. Teach it back

- Without the book/video open, explain pandas profiling before transformation in 60-90 seconds: what one input row/response represents,
- the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-04 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-05

DE05-05 - Cleaning with pandas and rejected rows

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Clean with pandas while preserving rejected rows and explicit reasons.
- TEACH IT TO YOURSELF
- A production-minded cleaning step should separate accepted and rejected data. Bad rows are evidence, not trash. Attach reason codes
- rather than silently dropping.
- WORKED DEMONSTRATION
- Normalize safe text, parse dates/numbers with explicit error handling, validate qty/price rules, and write accepted/rejected outputs.

3. Prerequisite / starting state

Prerequisite: DE05-04 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 4:47:03-5:08:08 - pandas cleaning and analysis. 1.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 4:47:03-5:08:08 - pandas cleaning and analysis. 1. STARTING STATE Prerequisite: DE05-04.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56-5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Process orders_bad.csv so every invalid row gets one or more reason codes. Process orders_bad.csv so every invalid row gets one or more reason codes.

8. Expected result / what you should see

- All source rows accounted for; rejects retain source fields + reason; no bad row silently disappears.
- COMMON FAILURE / RECOVER
- Failure to avoid: dropna() or errors="coerce" followed by filtering can hide defects if rejects are not preserved. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Split shipments_retest.csv into accepted and rejected rows with explicit reason codes. Prove input_rows = accepted_rows +
- rejected_rows and retain original fields.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

On a changed bad batch, produce accepted + rejected datasets and reconcile `source_rows = accepted_rows + rejected_rows`.

13. Validate

- All source rows accounted for; rejects retain source fields + reason; no bad row silently disappears.
- COMMON FAILURE / RECOVER
 - Failure to avoid: `dropna()` or `errors="coerce"` followed by filtering can hide defects if rejects are not preserved. Recovery: stop publishing,
 - restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Split `shipments_retest.csv` into accepted and rejected rows with explicit reason codes. Prove `input_rows = accepted_rows + rejected_rows` and retain original fields.

14. Teach it back

- Without the book/video open, explain cleaning with pandas and rejected rows in 60-90 seconds: what one input row/response represents,
- the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
 - Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
 - save evidence. A percentage score alone cannot mark DE05-05 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-06

DE05-06 - pandas merge, cardinality and aggregation

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Merge pandas tables with cardinality checks and aggregate without fan-out.
- TEACH IT TO YOURSELF
- Before merge, state left/right grain and prove uniqueness on the intended one side. `validate=` in pandas merge can enforce expected
 - cardinality.
- WORKED DEMONSTRATION
- Merge orders to customers on `customer_id` with `validate="many_to_one"`; compare row count and unmatched keys; aggregate accepted
 - sales by region.

3. Prerequisite / starting state

Prerequisite: DE05-05 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 7:15:11-7:34:04 - merge and concat. Teaching Book adds cardinality validation and fan-out controls.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_SUPPORT: Luke Barousse - Python for Data Analytics 7:15:11-7:34:04 - merge and concat. Teaching Book adds cardinality validation and fan-out controls. 1.

6. Useful video / source

Source: Luke Barousse — pandas / Python mechanics refresher

Assigned scope: 4:22:56-5:08:26

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Run the valid merge, then deliberately duplicate a customer key and observe the validation failure. Run the valid merge, then deliberately duplicate a customer key and observe the validation failure.

8. Expected result / what you should see

- Pre/post row counts preserved when expected; duplicate one-side key blocks the merge; unmatched count is explicit; grouped total
- reconciles.
- COMMON FAILURE / RECOVER
- Failure to avoid: A merge can run and still multiply money if the right-side key is not unique. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED

- Attempt many-to-one merge of shipment rows to carriers_retest.csv. Detect the deliberate duplicate carrier key before trusting totals, then
- separately report unmatched carrier IDs. Do not hide either issue with drop_duplicates().

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Use a changed dimension with one missing and one duplicated key; diagnose both without using drop_duplicates as a blind fix.

13. Validate

- Pre/post row counts preserved when expected; duplicate one-side key blocks the merge; unmatched count is explicit; grouped total
- reconciles.
- COMMON FAILURE / RECOVER
- Failure to avoid: A merge can run and still multiply money if the right-side key is not unique. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Attempt many-to-one merge of shipment rows to carriers_retest.csv. Detect the deliberate duplicate carrier key before trusting totals, then
- separately report unmatched carrier IDs. Do not hide either issue with drop_duplicates().

14. Teach it back

- Without the book/video open, explain pandas merge, cardinality and aggregation in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-06 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-07

DE05-07 - Data validation: record, batch and relationship rules

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Implement record, batch and relationship validation as explicit contracts.
- TEACH IT TO YOURSELF
- Validation has layers: record field/type/range rules; batch row/key/freshness rules; relationship rules against reference data. Collect
- actionable failures before publish.
- WORKED DEMONSTRATION
- Define an OrderRecord Pydantic model for record shape and add batch uniqueness + customer relationship checks outside the model.

3. Prerequisite / starting state

Prerequisite: DE05-06 mastered with saved evidence.

4. Picture it in your head

rules ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Complete Pydantic Data Validation Course 03:39-21:46 basic models/errors; 31:49-49:05 field constraints; 49:06-1:00:09 custom validators; 1:00:10-1:11:57 nested models. Teaching Book adds batch and relationship rules.

5. New words before we use them

rules ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Complete Pydantic Data Validation Course 03:39-21:46 basic models/errors; 31:49-49:05 field constraints; 49:06-1:00:09 custom validators; 1:00:10-1:11:57 nested models. Teaching Book adds batch and relationship rules.
1.

6. Useful video / source

Source: Corey Schafer — Complete Pydantic Data Validation Course

Assigned scope: Validation fundamentals → field/custom/nested validation; use only sections relevant to pipeline records

How to use it: Use this modern validation class for the mental model, then implement the course record/batch/relationship validation rules independently.

Note: Published 2025; directly relevant to runtime data validation and pipelines.

[Open source](#)

7. Follow with me once

Validate good data and a packaged bad batch. Produce reason-coded error output. Validate good data and a packaged bad batch. Produce reason-coded error output.

8. Expected result / what you should see

- Good data passes; each controlled defect is detected; validation report identifies row/key/rule; publish is blocked on critical failure.
- COMMON FAILURE / RECOVER
- Failure to avoid: Using type hints alone and assuming runtime values are validated automatically. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide

- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Create ShipmentRecord validation plus batch uniqueness and carrier relationship checks. Prove the packaged invalid date, zero units,
- negative unit value, missing carrier, duplicate shipment ID and invalid status are surfaced by named rules.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Add a new rule (allowed status or max future date) and prove a violating record is rejected while valid records still pass.

13. Validate

- Good data passes; each controlled defect is detected; validation report identifies row/key/rule; publish is blocked on critical failure.
- COMMON FAILURE / RECOVER
- Failure to avoid: Using type hints alone and assuming runtime values are validated automatically. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Create ShipmentRecord validation plus batch uniqueness and carrier relationship checks. Prove the packaged invalid date, zero units,
- negative unit value, missing carrier, duplicate shipment ID and invalid status are surfaced by named rules.

14. Teach it back

- Without the book/video open, explain data validation: record, batch and relationship rules in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-07 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-08

DE05-08 - HTTP and REST APIs from zero

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Explain HTTP/REST request-response basics using a safe local training API.
- TEACH IT TO YOURSELF
- HTTP exchanges a request for a response. Method, URL/path, query parameters, headers, status and body are separate parts.
- 2xx/4xx/5xx categories guide handling.
- WORKED DEMONSTRATION
- Start `mock_api_server.py`. Inspect `/health` and `/orders` with `curl`/browser/requests. Record status, headers/body shape.

3. Prerequisite / starting state

Prerequisite: DE05-07 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Python Requests Tutorial 00:00-25:00 full focused Requests lesson for request/response, GET, parameters, JSON and authentication context. Use the bundled local API for deterministic practice.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Python Requests Tutorial 00:00-25:00 full focused Requests lesson for request/response, GET, parameters, JSON and authentication context. Use the bundled local API for deterministic practice. SECONDARY SUPPORT - OPTIONAL OFFICIAL_SECONDARY: MDN HTTP Overview - Request/response structure, methods and status-code classes [open] 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Call `GET /health` and `GET /orders?page=1&per_page=4`; explain path vs query parameter. Call `GET /health` and `GET /orders?page=1&per_page=4`; explain path vs query parameter.

8. Expected result / what you should see

- 200 payload shape is understood; 404 is not parsed as valid orders; 500 is classified as server failure.
- COMMON FAILURE / RECOVER
- Failure to avoid: Calling `response.json()` and using the result before checking status/contract.
Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.

- FRESH CHANGED RETRY IF ASSIGNED
- Start the bundled local API. Call /health, /shipments?page=1&per_page=3, /missing and /shipments?fail=1. Record
- method/path/query/status and classify continue/stop/retry behavior.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Call /missing and /orders?fail=1; classify each result and state whether pipeline should retry, stop or treat as valid.

13. Validate

- 200 payload shape is understood; 404 is not parsed as valid orders; 500 is classified as server failure.
- COMMON FAILURE / RECOVER
- Failure to avoid: Calling response.json() and using the result before checking status/contract.
Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Start the bundled local API. Call /health, /shipments?page=1&per_page=3, /missing and /shipments?fail=1. Record
- method/path/query/status and classify continue/stop/retry behavior.

14. Teach it back

- Without the book/video open, explain http and rest apis from zero in 60-90 seconds: what one input row/response represents, the critical
- failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-08 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-09

DE05-09 - Calling APIs safely with requests

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Call APIs safely with requests using timeout, status handling and response validation.
- TEACH IT TO YOURSELF
- Network calls can fail slowly or return invalid payloads. A safe client uses timeouts, status handling, schema checks and bounded useful
- logging.
- WORKED DEMONSTRATION
- requests.Session/get with timeout; raise_for_status; parse JSON; validate expected keys before returning records.

3. Prerequisite / starting state

Prerequisite: DE05-08 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: Requests 2.34.2 Quickstart Make a Request; Passing Parameters; JSON Response Content; Response Status Codes; Timeouts; Errors and Exceptions. You already watched the visual foundation in DE05-08.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: Requests 2.34.2 Quickstart Make a Request; Passing Parameters; JSON Response Content; Response Status Codes; Timeouts; Errors and Exceptions. You already watched the visual foundation in DE05-08. SECONDARY SUPPORT - OPTIONAL OFFICIAL_SECONDARY: Requests Advanced Usage - Sessions and request behavior [open] 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Implement fetch_page() for local API and test 200/404/500 behavior. Implement fetch_page() for local API and test 200/404/500 behavior.

8. Expected result / what you should see

- Timeout configured; non-2xx does not enter normal processing; payload contract failures are distinct from HTTP failures.
- COMMON FAILURE / RECOVER
- Failure to avoid: except Exception: return [] turns outages into false “zero records” success. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.

- FRESH CHANGED RETRY IF ASSIGNED
- Implement a safe page fetch for /shipments with timeout, status check and required payload keys. Then call /shipments?bad_payload=1
- and prove it fails as a payload-contract error rather than returning zero rows.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Point fetch_page at a changed endpoint response missing expected items; fail clearly rather than returning empty success.

13. Validate

- Timeout configured; non-2xx does not enter normal processing; payload contract failures are distinct from HTTP failures.
- COMMON FAILURE / RECOVER
- Failure to avoid: except Exception: return [] turns outages into false “zero records” success. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Implement a safe page fetch for /shipments with timeout, status check and required payload keys. Then call /shipments?bad_payload=1
- and prove it fails as a payload-contract error rather than returning zero rows.

14. Teach it back

- Without the book/video open, explain calling apis safely with requests in 60-90 seconds: what one input row/response represents, the
- critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-09 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-10

DE05-10 - Pagination and authentication without leaking secrets

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Handle pagination and authentication without leaking secrets.
- TEACH IT TO YOURSELF
- Pagination is state: page/cursor advances only after a successful validated response. Authentication belongs in environment/secret
- storage, never source code or screenshots.
- WORKED DEMONSTRATION
- Use API_TOKEN environment variable; send Authorization header; loop next_page until None; collect IDs and prove no duplicates/gaps.

3. Prerequisite / starting state

Prerequisite: DE05-09 mastered with saved evidence.

4. Picture it in your head

secrets ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: Requests 2.34.2 Authentication Basic Authentication and custom authentication concepts. Deterministic pagination and secret hygiene are course-built against the bundled local API.

5. New words before we use them

secrets ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: Requests 2.34.2 Authentication Basic Authentication and custom authentication concepts. Deterministic pagination and secret hygiene are course-built against the bundled local API. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Fetch all local API pages with TRAINING_TOKEN supplied via environment. Save page/count evidence, not the token. Fetch all local API pages with TRAINING_TOKEN supplied via environment. Save page/count evidence, not the token.

8. Expected result / what you should see

- All expected IDs fetched once; token absent from source/log; auth failure stops clearly; pagination termination is deterministic.
- COMMON FAILURE / RECOVER
- Failure to avoid: Hard-coding bearer token into Python or logging full Authorization headers. Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide

- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Fetch all /shipments pages using API_TOKEN from the environment. Repeat with per_page=2 and per_page=4; the ordered unique
- shipment ID set must match. Prove the token is absent from code/log evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Change per_page and repeat; total IDs must remain the same. Then test missing/invalid token and prove no secret is logged.

13. Validate

- All expected IDs fetched once; token absent from source/log; auth failure stops clearly; pagination termination is deterministic.
- COMMON FAILURE / RECOVER
- Failure to avoid: Hard-coding bearer token into Python or logging full Authorization headers.
Recovery: stop publishing, restate the
- input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not hide
- a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Fetch all /shipments pages using API_TOKEN from the environment. Repeat with per_page=2 and per_page=4; the ordered unique
- shipment ID set must match. Prove the token is absent from code/log evidence.

14. Teach it back

- Without the book/video open, explain pagination and authentication without leaking secrets in 60-90 seconds: what one input
- row/response represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-10 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-11

DE05-11 - Retries, rate limits and partial failure

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Design bounded retries for rate limits/transient failures and preserve partial-failure evidence.
- TEACH IT TO YOURSELF
- Retry only failures that may recover, bound attempts, honor Retry-After/backoff, and never retry permanent data-contract errors blindly.
- WORKED DEMONSTRATION
- Implement retry wrapper for 429/5xx with max attempts and small training backoff. Save attempt/status log. Do not retry
- 400/401/validation failures.

3. Prerequisite / starting state

Prerequisite: DE05-10 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_REFERENCE: Requests 2.34.2 Advanced Usage Sessions; Transport Adapters; Example: Automatic Retries; Timeouts. Retry policy is failure-injected locally in this course.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_REFERENCE: Requests 2.34.2 Advanced Usage Sessions; Transport Adapters; Example: Automatic Retries; Timeouts. Retry policy is failure-injected locally in this course. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Use local ?rate_limit=1 and ?fail_once=1 training cases; observe bounded recovery/failure. Use local ?rate_limit=1 and ?fail_once=1 training cases; observe bounded recovery/failure.

8. Expected result / what you should see

- Retries bounded; 429/5xx policy explicit; partial completion state distinguishable from full success; no infinite loop.
- COMMON FAILURE / RECOVER
- Failure to avoid: Blind retry around all exceptions can hammer a service and conceal permanent bad data. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED

- Use `/shipments?rate_limit_once=1` to prove one bounded recovery, then `/shipments?fail=1` to prove a permanent failure is not mislabeled
- full success. Save attempt count and last-safe state.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Simulate a page that fails permanently after earlier pages succeeded; preserve last safe page and do not falsely mark full batch complete.

13. Validate

- Retries bounded; 429/5xx policy explicit; partial completion state distinguishable from full success; no infinite loop.
- COMMON FAILURE / RECOVER
- Failure to avoid: Blind retry around all exceptions can hammer a service and conceal permanent bad data. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Use `/shipments?rate_limit_once=1` to prove one bounded recovery, then `/shipments?fail=1` to prove a permanent failure is not mislabeled
- full success. Save attempt count and last-safe state.

14. Teach it back

- Without the book/video open, explain retries, rate limits and partial failure in 60-90 seconds: what one input row/response represents, the
- critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-11 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-12

DE05-12 - Python + PostgreSQL with SQLAlchemy

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Use SQLAlchemy to connect Python to the already-installed PostgreSQL lab safely.
- TEACH IT TO YOURSELF
- SQLAlchemy manages engine/connection and parameterized execution; PostgreSQL was first installed/verified in M03. M05 teaches
- Python integration, not database modeling.
- WORKED DEMONSTRATION
- Read DATABASE_URL from environment; create_engine; run SELECT version()/simple staging table; insert accepted rows using
- parameterized SQL/SQLAlchemy Core; transaction rollback on controlled failure.

3. Prerequisite / starting state

Prerequisite: DE05-11 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: SQLAlchemy 2.0 Unified Tutorial Establishing Connectivity - the Engine; Working with Transactions and the DBAPI; Working with Data. No Docker execution in M05.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE OFFICIAL_PRIMARY: SQLAlchemy 2.0 Unified Tutorial Establishing Connectivity - the Engine; Working with Transactions and the DBAPI; Working with Data. No Docker execution in M05. SECONDARY SUPPORT - OPTIONAL OFFICIAL_OPTIONAL_VISUAL: SQLAlchemy Library - 2.0 tutorial by Mike Bayer - Optional visual for Engine/Connection/transaction concepts after the assigned 2.0 docs [open] 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Load a small accepted batch into a dedicated m05 schema/table, then query row count and total; rerun with stable key/upsert or clean-reset strategy stated explicitly. Load a small accepted batch into a dedicated m05 schema/table, then query row count and total; rerun with stable key/upsert or clean-reset strategy stated explicitly.

8. Expected result / what you should see

- No credential in code; connection works; row/key/totals reconcile; failure rolls back; rerun policy is explicit.
- COMMON FAILURE / RECOVER
- Failure to avoid: Embedding postgres password in source or assuming commit happened because execute() returned. Recovery: stop

- publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent
- validation. Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Load the accepted retest shipment batch to a separate m05_retest table/run_id using SQLAlchemy and the existing M03 PostgreSQL
- install. Inject one transaction failure and prove rollback leaves no half-published batch.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Load a changed batch using a separate run_id. Prove a failed transaction does not leave half-published rows.

13. Validate

- No credential in code; connection works; row/key/totals reconcile; failure rolls back; rerun policy is explicit.
- COMMON FAILURE / RECOVER
- Failure to avoid: Embedding postgres password in source or assuming commit happened because execute() returned. Recovery: stop
- publishing, restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent
- validation. Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Load the accepted retest shipment batch to a separate m05_retest table/run_id using SQLAlchemy and the existing M03 PostgreSQL
- install. Inject one transaction failure and prove rollback leaves no half-published batch.

14. Teach it back

- Without the book/video open, explain python + postgresql with sqlalchemy in 60-90 seconds: what one input row/response represents, the
- critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-12 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-13

DE05-13 - Configuration, environment variables and secret hygiene

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Separate configuration, environment-specific values and secrets from logic.
- TEACH IT TO YOURSELF
- Config is data controlling behavior. Non-secret defaults/examples can live in repo; secrets come from environment/secret manager and
- must never be committed.
- WORKED DEMONSTRATION
- Load config.example.json; override selected values with env vars; validate required settings; redact secrets in logs.

3. Prerequisite / starting state

Prerequisite: DE05-12 mastered with saved evidence.

4. Picture it in your head

hygiene ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Python: Using Environment Variables Full focused Windows environment-variable lesson. Teaching Book adds config layering and redaction.

5. New words before we use them

hygiene ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Python: Using Environment Variables Full focused Windows environment-variable lesson. Teaching Book adds config layering and redaction. 1.

6. Useful video / source

Source: Luke Barousse — environments / project setup

Assigned scope: 5:50:56–6:27:15

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Override output directory and DATABASE_URL via environment without editing Python. Override output directory and DATABASE_URL via environment without editing Python.

8. Expected result / what you should see

- Same code runs with two configs; missing required setting fails clearly; secrets are excluded from Git/log evidence.
- COMMON FAILURE / RECOVER
- Failure to avoid: A .env file is convenient locally but is not safe to commit just because it is hidden in Explorer. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED

- Create a second non-secret config profile for shipments with a different output folder/batch size. Override the fake training token through
- the environment without editing pipeline logic; scan evidence for secret leakage.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Create a second config profile with different batch size/output path; prove code unchanged and secret value never printed.

13. Validate

- Same code runs with two configs; missing required setting fails clearly; secrets are excluded from Git/log evidence.
- COMMON FAILURE / RECOVER
- Failure to avoid: A .env file is convenient locally but is not safe to commit just because it is hidden in Explorer. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Create a second non-secret config profile for shipments with a different output folder/batch size. Override the fake training token through
- the environment without editing pipeline logic; scan evidence for secret leakage.

14. Teach it back

- Without the book/video open, explain configuration, environment variables and secret hygiene in 60-90 seconds: what one input
- row/response represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-13 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-14

DE05-14 - Logging, testing and reusable pipeline code

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Combine structured logging, tests and reusable pipeline code around clear boundaries.
- TEACH IT TO YOURSELF
- Reliable code separates pure transformations from I/O, logs run context/counts, and tests both small functions and integrated behavior.
- WORKED DEMONSTRATION
- Refactor into read/validate/transform/write functions; add pytest unit tests and one temp-directory integration test; structured log contains
- run_id/source/counts/status.

3. Prerequisite / starting state

Prerequisite: DE05-13 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Logging Basics Full focused logging lesson. The Teaching Book also exposes the current pytest documentation as secondary support for regression tests.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE VIDEO_PRIMARY: Corey Schafer - Logging Basics Full focused logging lesson. The Teaching Book also exposes the current pytest documentation as secondary support for regression tests. SECONDARY SUPPORT - OPTIONAL OFFICIAL_SECONDARY: pytest documentation - Writing/running tests and logging capture [open] 1.

6. Useful video / source

Source: PyCon US 2025 — Building Resilient Data Pipelines: The Power of Idempotency
Assigned scope: Full ~25-minute talk; focus on resilient pipeline design + pytest strategy
How to use it: Use this as the testing/reliability visual layer. The Teaching Book still teaches logging and reusable code structure. Close the video before the independent pipeline task.
Note: PyCon US 2025 talk explicitly covers data-pipeline design and testing with pytest; it supplements rather than replaces the course logging/reuse lab.

[Open source](#)

7. Follow with me once

Run tests, then inject one bad record and confirm the expected test/failure evidence. Run tests, then inject one bad record and confirm the expected test/failure evidence.

8. Expected result / what you should see

- Tests fail before repair and pass after; logs identify run and counts; functions have clear responsibilities; failures propagate.
- COMMON FAILURE / RECOVER
- Failure to avoid: A green test suite that never checks bad input/rerun behavior gives false confidence. Recovery: stop publishing, restate
- the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not

- hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Write a regression test for one shipment validation/logging bug, capture the failing test first, repair code, then capture the passing test.
- Log run_id, source, accepted/rejected counts and final status.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Add one regression test for a bug you deliberately create and repair. Explain what the test prevents.

13. Validate

- Tests fail before repair and pass after; logs identify run and counts; functions have clear responsibilities; failures propagate.
- COMMON FAILURE / RECOVER
- Failure to avoid: A green test suite that never checks bad input/rerun behavior gives false confidence. Recovery: stop publishing, restate
- the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation. Do not
- hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Write a regression test for one shipment validation/logging bug, capture the failing test first, repair code, then capture the passing test.
- Log run_id, source, accepted/rejected counts and final status.

14. Teach it back

- Without the book/video open, explain logging, testing and reusable pipeline code in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-14 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: DE05-15

DE05-15 - Integrated Python data-engineering pipeline

1. Why it matters

This matters because later Python for Data Engineering work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

- Build and defend an integrated Python ingestion pipeline with safe reruns and PostgreSQL/output evidence.
- TEACH IT TO YOURSELF
- The final module proof combines preserve raw -> profile -> validate/quarantine -> transform -> API/file ingestion -> safe state -> optional
- PostgreSQL load -> logs/tests -> reconciliation.
- WORKED DEMONSTRATION
- Use M05_PIPELINE_TODO.py as blank-slate scaffold. The lesson book explains architecture, not a finished copy-paste solution.

3. Prerequisite / starting state

Prerequisite: DE05-14 mastered with saved evidence.

4. Picture it in your head

ASSIGNED VIDEO / SUPPORT ROUTE PRACTICE_ONLY: Independent integrated checkpoint - no tutorial video by design Build from the saved learner attempt. External references stay closed until a genuine attempt exists.

5. New words before we use them

ASSIGNED VIDEO / SUPPORT ROUTE PRACTICE_ONLY: Independent integrated checkpoint - no tutorial video by design Build from the saved learner attempt. External references stay closed until a genuine attempt exists. 1.

6. Useful video / source

Source: DataTalksClub — Docker/PostgreSQL data-engineering workshop

Assigned scope: Use the matching ingestion / Python-script / PostgreSQL chapter

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

Note: Use only the matching chapter; our Teaching Book adds retries, validation, idempotency, secrets and fresh independent work.

[Open source](#)

7. Follow with me once

Build one file-source run and one API-source run. Save source receipt, accepted/rejected counts, output/database controls, log and rerun result. Build one file-source run and one API-source run. Save source receipt, accepted/rejected counts, output/database controls, log and rerun result.

8. Expected result / what you should see

- PASS only when all integrated pipeline contracts are evidenced: raw/source receipt preserved; accepted + rejected reconcile to input; no
- silent bad-data publish; rerun creates no duplicate publish; transient failure follows bounded recovery; permanent/partial failure is not
- labeled full success; no secret appears in code/log/evidence; output or PostgreSQL state reconciles; tests/logs support the run; and

- another learner could reproduce the run from README/config. Any unresolved critical contract => TARGETED REPAIR.
- COMMON FAILURE / RECOVER
- Failure to avoid: Treating “script exited 0” as sufficient without proving data correctness, state and recovery. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Run the complete changed shipments pipeline from raw receipt + local API through validation/quarantine, transformation, bounded retry
- state, optional PostgreSQL load, logs/tests and reconciliation. Preserve the original practical attempt; this fresh run is separate evidence.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

Common failure patterns: copying without understanding, changing more than one thing at once, trusting a plausible output without an independent control, or deleting the failed attempt instead of learning from it.

11. If you get stuck

Stop at the first point where your result diverges. Restate the intended input/state and expected output, inspect the smallest relevant step, reproduce only that step, and keep the failed attempt as evidence. Open protected review only after a genuine attempt has been saved.

12. Small independent task

Run a fresh changed batch with one duplicate, one invalid row and one transient API failure. Recover without corrupting previously published data.

13. Validate

- PASS only when all integrated pipeline contracts are evidenced: raw/source receipt preserved; accepted + rejected reconcile to input; no
- silent bad-data publish; rerun creates no duplicate publish; transient failure follows bounded recovery; permanent/partial failure is not
- labeled full success; no secret appears in code/log/evidence; output or PostgreSQL state reconciles; tests/logs support the run; and
- another learner could reproduce the run from README/config. Any unresolved critical contract => TARGETED REPAIR.
- COMMON FAILURE / RECOVER
- Failure to avoid: Treating “script exited 0” as sufficient without proving data correctness, state and recovery. Recovery: stop publishing,
- restate the input/output contract, inspect the smallest failing control, repair only that competency, then rerun the independent validation.
- Do not hide a defect by dropping data or returning an empty result.
- FRESH CHANGED RETRY IF ASSIGNED
- Run the complete changed shipments pipeline from raw receipt + local API through validation/quarantine, transformation, bounded retry

- state, optional PostgreSQL load, logs/tests and reconciliation. Preserve the original practical attempt; this fresh run is separate evidence.

14. Teach it back

- Without the book/video open, explain integrated python data-engineering pipeline in 60-90 seconds: what one input row/response
- represents, the critical failure mode, the validation control, and what evidence proves success.
- MASTER / FINISHED WHEN
- Master only when you can reproduce the competency on the independent or assigned changed task, satisfy the printed validation, and
- save evidence. A percentage score alone cannot mark DE05-15 mastered.

15. Protected review + changed retry

Save the genuine independent attempt first. Only then may the matching protected review open. Diagnose the exact weak competency, close the review, and complete a changed retry without the answer visible. The corrected-with-review attempt is repair evidence, not mastery.

16. Evidence / mastery + exact next route

Complete the matching guided/independent work with review material closed; save the actual result/error and validation evidence. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.